python线程

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

目录

Contents

- 线程的介绍

  线程概念、作用

- 多线程完成多任务

  多线程步骤、无参线程、有参线程

- 线程的注意点

  执行顺序、共享全局变量、数据安全

- 互斥锁的使用

  概念、api使用步骤、死锁

- 进程和线程对比

  关系对比、区别对比、优缺点

目录

Contents

- 线程的介绍

  线程概念、作用

- 多线程完成多任务
- 线程的注意点
- 互斥锁的使用
- 进程和线程对比

学习目标

Learning Objectives

1. 知道线程是什么？
2. 知道线程的作用

线程的介绍

在Python中，想要实现多任务除了使用进程，还可以使用线程来完成。

线程的概念

进程是分配资源的基本单位，一旦创建一个进程就会分配一定的资源；

线程是cpu调度的基本单位，每个进程至少都有一个线程，而这个线程就是我们通常说的主线程。

线程的介绍

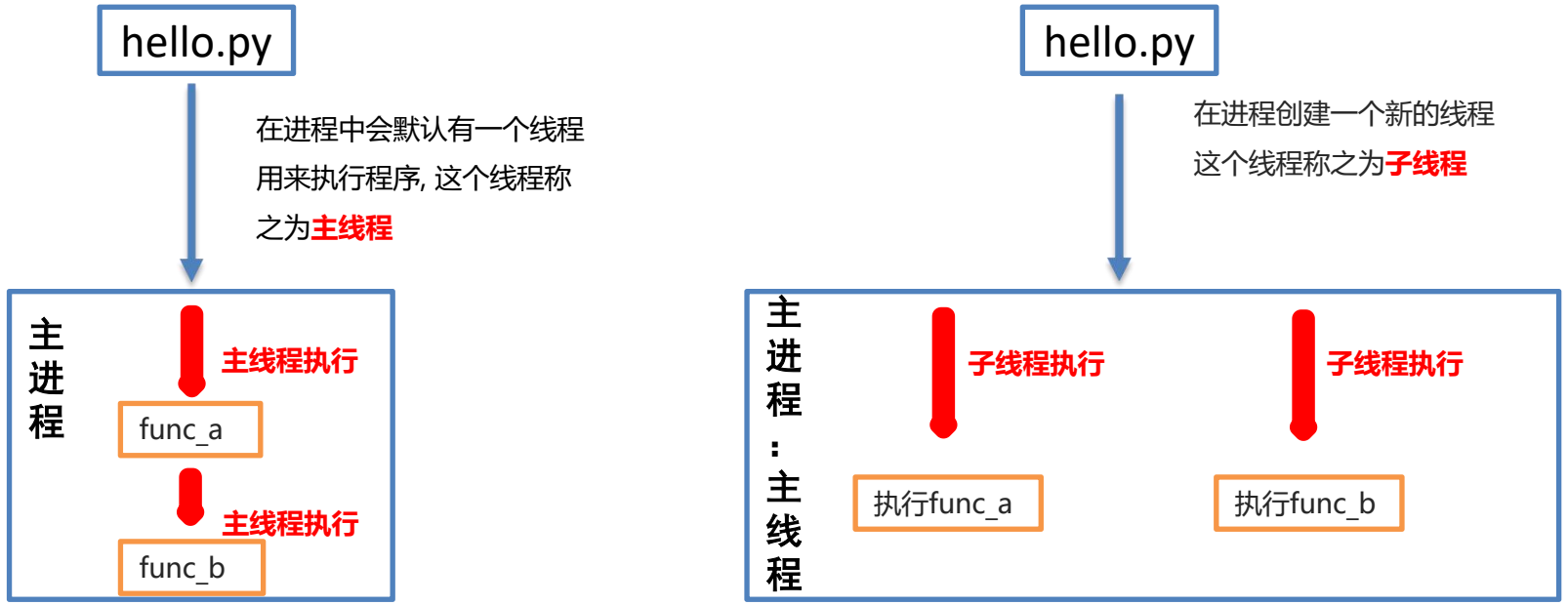
hello.py

```python
def func_a():
    print("任务A")

def func_b():
    print("任务B")

func_a()
func_b()
```

思考:

图中是一个非常简单的程序，一旦运行hello.py这个程序，按照代码的执行顺序，func_a函数执行完毕后才能执行func_b函数．如果可以让func_a和func_b同时运行，显然执行hello.py这个程序的效率会大大提升．

线程的作用



线程用来实现多任务编程

总结

1、什么是线程？

线程依附于进程执行，是CPU调度的基本单元

2、线程的作用是什么？

线程用来实现多任务编程

目录

Contents

- 线程的介绍
- 多线程完成多任务

  多线程步骤、无参线程、有参线程
- 线程的注意点
- 互斥锁的使用
- 进程和线程对比

学习目标

Learning Objectives

1. 知道使用多线程完成多任务步骤
2. 能够编程实现多线程完成多任务
3. 知道线程任务的传参方式并实现

线程创建的步骤

1. 导入线程模块

   import threading

2. 通过线程类创建线程对象

   线程对象 = **threading.Thread**(target=任务名)

3. 启动线程执行任务

   线程对象.start()

线程类参数说明

线程对象=threading.Thread([group [, target [, name [, args [, kwargs]]]]])

group：线程组，目前只能使用None

target：执行的目标任务名

args：以元组的方式给执行任务传参，元组方式传参一定要和目标任务函数参数的顺序保持一致。

kwargs：以字典方式给执行任务传参，字典方式传参字典中的key一定要和参数的顺序保持一致

name：线程名，一般不用设置

多线程完成多任务的代码

案例分析

例如，使用多线程来模拟一边写代码，一边听音乐的功能。

```
#1. 导入线程工具包
import threading
import time
# 编写代码任务
def coding():
    for i in range(3):
        print("coding...")
        time.sleep(0.2)
# 听音乐任务
def music():
    for i in range(3):
        print("music...")
        time.sleep(0.2)
```

```
if __name__ == '__main__':
    # 2.创建子线程
    coding_thread = threading.Thread(target=coding)
    music_thread = threading.Thread(target=music)
    # 3.启动子线程执行任务
    coding_thread.start()
    music_thread.start()
```

线程带参数的任务

案例分析

使用多线程来模拟小明一边编写num行代码，一边听count首音乐功能实现。

```
# 1.导入线程工具包
import threading
import time
# 编写代码任务
def coding(num,name):
    for i in range(num):
        print(f"{name}正在编写第{i}行代码...")
        time.sleep(0.2)
# 听音乐任务
def music(count,name):
    for i in range(count):
        print(f"{name}正在听第{i}首歌...")
        time.sleep(0.2)
```

```
if __name__ == '__main__':
    # 2.创建子线程
    coding_thread = threading.Thread(target=coding, args=(3,'小明'))
    music_thread = threading.Thread(target=music, kwargs={'name':'小明
','count' : 3})
    # 3.启动子线程执行任务
    coding_thread.start()
    music_thread.start()
```

总结

1. 创建线程的流程是什么？

   a. 导入线程模块

   ```
   import threading
   ```

   b. 创建子线程并指定执行的任务

   ```
   sub_thread = threading.Thread(target=任务名)
   ```

   c. 启动线程执行任务

   ```
   sub_thread.start()
   ```

2. 线程传参的两种方式是什么？

   a .元组方式传参： 元组方式传参一定要和参数的顺序保持一致。

   b.字典方式传参：字典方式传参字典中的key一定要和参数名保持一致

目录

Contents

- 线程的介绍
- 多线程完成多任务
- 线程的注意点

  执行顺序、共享全局变量、数据安全
- 互斥锁的使用
- 进程和线程对比

学习目标

Learning Objectives

1. 知道线程的执行顺序
2. 知道线程间能够共享全局变量
3. 知道多线程的数据安全问题

线程的注意点介绍

1. 线程之间执行是无序的
2. 主线程会等待所有的子线程执行结束再结束
3. 线程之间共享全局变量
4. 线程之间共享全局变量数据出现错误问题

线程之间执行是无序的

线程之间执行是无序的，它是由cpu调度决定的 ，cpu调度哪个线程，哪个线程就执行，没有调度的线程是不能执行的。

案例分析

创建多个线程，多次运行，观察各次线程的执行顺序

```
# 1.导入线程工具包
import threading
import time
# 获取线程信息函数
def get_info():
    time.sleep(0.5)
    # 获取线程信息并打印
    current_thread = threading.current_thread()
    print(current_thread)
```

```
if __name__ == '__main__':
    # 2.创建多个子线程，并启动
    for i in range(10):
        sub_thread = threading.Thread(target=get_info)
        sub_thread.start()
```

进程之间执行也是无序的，它是由操作系统调度决定的，操作系统调度哪个进程，哪个进程就先执行，没有调度的进程不能执行

主线程会等待所有的子线程执行结束再结束

案例分析

假如创建一个子线程，这个子线程执行完大概需要2.5秒钟，现在让主线程执行1秒钟就退出程序，查看一下执行结果

```
# 1.导入线程工具包
import threading
import time
# 工作函数，执行2s钟
def work():
    for i in range(10):
        print("working...")
        time.sleep(0.2)
```

```
if __name__ == '__main__':
    # 2.创建子线程
    work_thread = threading.Thread(target=work)
    # 3.启动线程
    work_thread.start()
    # 延时一秒
    time.sleep(1)
    print("主线程执行完毕")
```

```
working...
主线程执行完毕
working...

Process finished with exit code 0
```

主线程会等待所有的子线程执行结束再结束

说明：
通过上面代码的执行结果，我们可以得知：**主线程会等待所有的子线程执行结束再结束**
假如我们就让主线程执行1秒钟，子线程就销毁不再执行，那怎么办呢？
我们可以设置**守护主线程**

守护主线程：
守护主线程就是主线程退出子线程销毁不再执行

设置守护主线程有两种方式：

```
threading.Thread(target=show_info, daemon=True)
线程对象.setDaemon(True)
```

设置守护进程的示例代码：

```python
# 1.导入线程工具包
import threading
import time

# 工作函数，执行2s钟
def work():
    for i in range(10):
        print("working...")
        time.sleep(0.2)
```

```python
if __name__ == '__main__':
    # 创建子线程
    # 方式1 参数方式设置守护主线程
    work_thread = threading.Thread(target=work, daemon=True)
    # 启动线程
    # 方式2
     # work_thread = threading.Thread(target=work)
    # work_thread.setDaemon(True)
    work_thread.start()
    # 延时一秒
    time.sleep(1)
    print("主线程执行完毕")
```

线程之间共享全局变量

案例分析

定义一个列表类型的全局变量，创建两个子线程分别执行向全局变量添加数据的任务和向全局变量读取数据的任务，查看线程之间是否共享全局变量数据

```
# 1.导入线程工具包
import threading
import time
# 2.定义全局变量
my_list = []
# 写入数据的任务
def write_data():
    for i in range(3):
        print("add:", i)
        my_list.append(i)
    print("write:", my_list)
# 读取数据的任务
def read_data():
    print("read:", my_list)
```

```
if __name__ == '__main__':
    # 3.创建子线程
    write_thread = threading.Thread(target=write_data)
    read_thread = threading.Thread(target=read_data)
    # 4、启动线程
    write_thread.start()
    # 延时一秒
    time.sleep(1)
    read_thread.start()
```

```
write: [0, 1, 2]
read: [0, 1, 2]
```

线程之间共享全局变量数据出现错误问题

案例分析

定义两个函数，实现循环100万次，每循环一次给全局变量加1，创建两个子线程执行对应的两个函数，查看计算后的结果

```
# 1.导入线程工具包
import threading
# 2.定义全局变量
g_num = 0
# 3.定义两个函数分别对全局变量加1，并打印最后的结果
def sum_num1():
    for i in range(1000000):
        global g_num
        g_num += 1
    print("sum1:", g_num)
def sum_num2():
    for i in range(1000000):
        global g_num
        g_num += 1
    print("sum2:", g_num)
```

```
if __name__ == '__main__':
    # 4.创建子线程
    sum1_thread = threading.Thread(target=sum_num1)
    sum2_thread = threading.Thread(target=sum_num2)

    # 5.启动线程
    sum1_thread.start()
    sum2_thread.start()
```

```
sum2: 1400904
sum1: 1405106
```

错误分析

两个线程first_thread和second_thread都要对全局变量g_num(默认是0)进行加1运算，但是由于是多线程同时操作，有可能出现下面情况：

1. 在g_num=0时，first_thread取得g_num=0。此时系统把first_thread调度为等待状态，把second_thread转换为”running”状态，t2也获得g_num=0
2. 然后second_thread对得到的值进行加1并赋给g_num，使得g_num=1
3. 然后系统又把second_thread调度为等待，把first_thread转为执行。线程t1又把它之前得到的0加1后赋值给g_num。
4. 这样导致虽然first_thread和first_thread都对g_num加1，但结果仍然是g_num=1

全局变量数据错误的解决办法:

线程同步：保证同一时刻只能有一个线程去操作全局变量
同步：就是协同步调，按预定的先后次序进行运行。如:你说完，我再说，好比现实生活中的对讲机

线程同步的方式：**锁**

总结

1. 线程执行执行是（）？ A 有序 B无序

2. 主线程默认会等待所有子线程执行结束再结束。设置（）的目的是主线程退出子线程销毁。

守护主进程，threading.Thread(target=show_info, daemon=True)或 线程对象.setDaemon(True)

3. 线程之间（）全局变量？。A 共享 B 不共享

4. 线程之间共享全局变量可能会导致数据出现错误问题，可以使用（）来解决这个问题。

线程同步

目录

Contents

- 线程的介绍
- 多线程完成多任务
- 线程的注意点
- 互斥锁的使用

  概念、api使用步骤、死锁
- 进程和线程对比

互斥锁

互斥锁: 对共享数据进行锁定，保证同一时刻只有一个线程去操作。

注意: 互斥锁是**多个线程一起去抢**，抢到锁的线程先执行，没有抢到锁的线程进行等待，等锁使用完释放后，其它等待的线程再去抢这个锁。

互斥锁的使用流程

1. 互斥锁的创建

mutex = threading.Lock()

2. 上锁

mutex.acquire()

3. 释放锁

mutex.release()

使用互斥锁保证线程间的数据安全

案例分析

定义两个函数，实现循环100万次，每循环一次给全局变量加1，创建两个子线程执行对应的两个函数，添加互斥锁后，查看计算后的结果

```
# 1.导入工具包
import threading
# 2.定义全局变量
g_num = 0
# 3.创建互斥锁
mutex = threading.Lock()
# 4.定义两个函数实现加法
def sum_num1():
    mutex.acquire() # 上锁
    for I in range(1000000):
        global g_num
        g_num += 1
    mutex.release() # 解锁
    print("g_num1:", g_num)
```

```
def sum_num2():
    mutex.acquire() # 上锁
    for i in range(1000000):
        global g_num
        g_num += 1
    mutex.release() # 解锁
    print("g_num2:", g_num)
if __name__ == '__main__':
    # 5.创建子线程
    sum1_thread = threading.Thread(target=sum_num1)
    sum2_thread = threading.Thread(target=sum_num2)
    # 6.启动线程
    sum1_thread.start()
    sum2_thread.start()
```

```
g_num1: 1000000
g_num2: 2000000
```

死锁

一直等待对方释放锁的情景就是死锁。

死锁的原因:

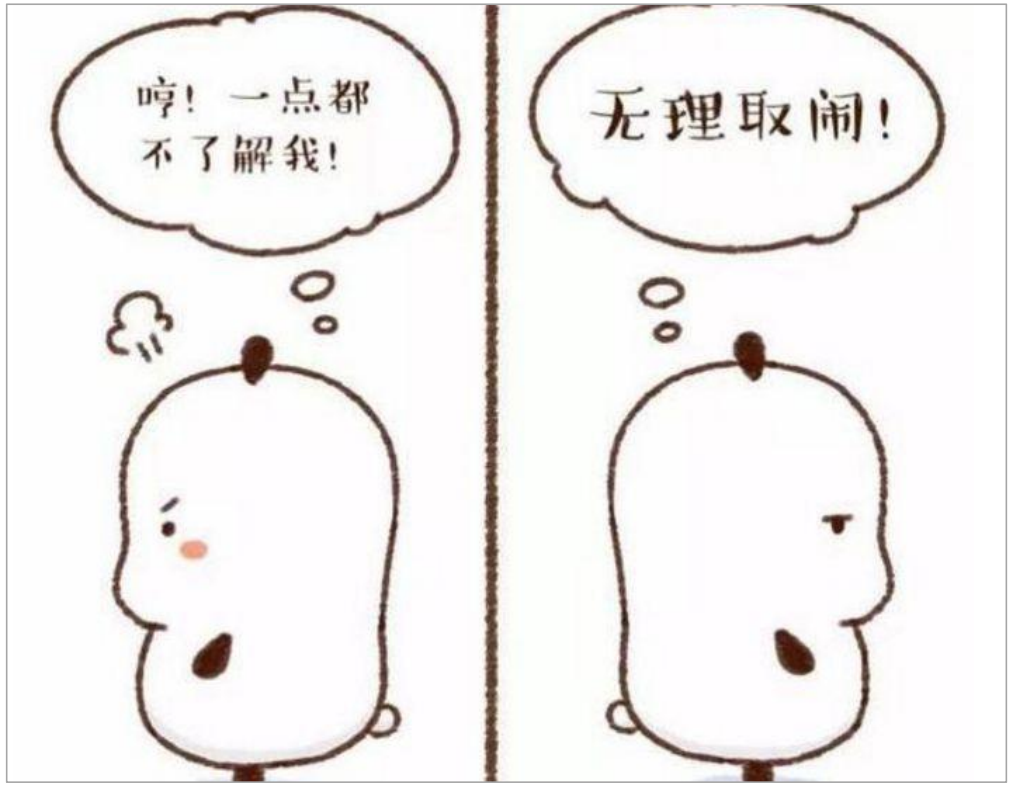
使用互斥锁的时候需要注意死锁的问题，

未在合适的地方注意释放锁

死锁的结果:

会造成应用程序的停止响应，

应用程序无法再继续往下执行了



死锁

案例分析

定义两个函数，实现循环100万次，每循环一次给全局变量加1，创建两个子线程执行对应的两个函数，死锁后，查看计算后的结果

```
# 1.导入工具包
import threading
# 2.定义全局变量
g_num = 0
# 3.创建互斥锁
mutex = threading.Lock()
# 4.定义两个函数实现加法
def sum_num1():
    mutex.acquire() # 上锁
    for I in range(1000000):
        global g_num
        g_num += 1
    print("g_num1:", g_num)
```

```
def sum_num2():
    mutex.acquire() # 上锁
    for i in range(1000000):
        global g_num
        g_num += 1
    print("g_num2:", g_num)
if __name__ == '__main__':
    # 5.创建子线程
    sum1_thread = threading.Thread(target=sum_num1)
    sum2_thread = threading.Thread(target=sum_num2)
    # 6.启动线程
    sum1_thread.start()
    sum2_thread.start()
```

总结

1. 互斥锁是什么？

对共享数据进行锁定，保证同一时刻只有一个线程去操作。

2. 互斥锁的使用步骤 ？

互斥锁的创建threading.Lock()，上锁mutex.acquire()，释放锁release

3. 死锁是什么？产生的原因？效果是什么？

一直等待对方释放锁的情景就是死锁。

原因:

使用互斥锁的时候需要注意死锁的问题，未在合适的地方注意释放锁

结果:

会造成应用程序的停止响应，应用程序无法再继续往下执行了

目录

Contents

- 线程的介绍
- 多线程完成多任务
- 线程的注意点
- 互斥锁的使用
- 进程和线程对比

关系对比、区别对比、优缺点

学习目标

Learning Objectives

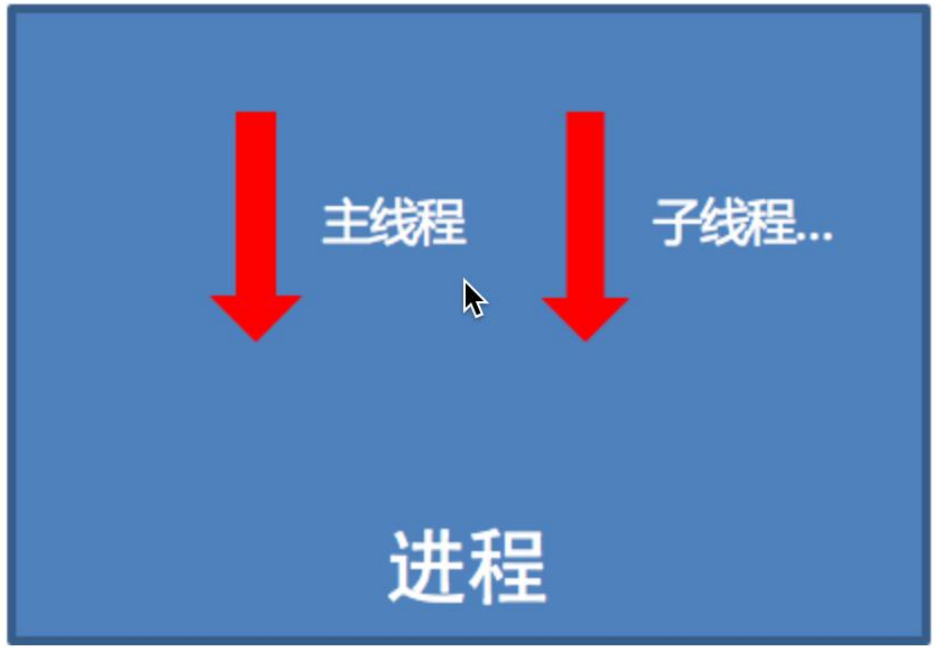
1. 能够知道进程和线程的关系

进程和线程的对比的三个方向

1. 关系对比
2. 区别对比
3. 优缺点对比

关系对比

1. 线程是依附在进程里面的，没有进程就没有线程
2. 一个进程默认提供一条线程，进程可以创建多个线程



区别对比

1. 进程之间不共享全局变量
2. 线程之间共享全局变量，但是要注意资源竞争的问题，解决办法：互斥锁
3. 创建进程的资源开销要比创建线程的资源开销要大
4. 进程是操作系统资源分配的基本单位，线程是CPU调度的基本单位
5. 线程不能够独立执行，必须依存在进程中
6. Python中多进程开发比单进程多线程开发稳定性要强

优缺点对比

进程优缺点：

- 优点：可以用多核
- 缺点：资源开销大

线程优缺点：

- 优点：资源开销小
- 缺点：不能使用多核

总结

1. 进程和线程都是完成（）的一种方式
2. 多进程要比多线程消耗的资源（多/少），但是多进程开发比单进程多线程开发稳定性要（强/弱），某个进程挂掉不会影响其它进程。
3. 多进程可以使用cpu的多核运行，多线程可以（）全局变量。
4. 线程不能单独执行必须依附在（）里面

答案：1 多任务　2 多 强 3 共享 4 进程

黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌